

Directives

Extending HTML with your own behavior — a Directive is a class that can change the **appearance or behavior** of an HTML element, without being a whole new Component.

1 · THE CONCEPT

What is a directive?

In fact, a Component *is* a directive — just one with its own template. Sometimes we don't want a whole new Component — we just want to **add** a small behavior to an existing element (show/hide it, change its color, repeat it). Goal: reuse a small piece of DOM logic across many elements without duplicating code.

2 · STRUCTURAL DIRECTIVES

Conditionally show or repeat elements

Modern (Angular 17+) control flow

```
@if (isLoggedIn) {  
  <p>Welcome back!</p>  
}  
  
@for (item of items; track item.id) {  
  <li>{{ item.name }}</li>  
}
```

Legacy syntax (still seen in older projects)

```
<p *ngIf="isLoggedIn">Welcome back!</p>  
<li *ngFor="let item of items">{{ item.name }}</li>
```

`@if` and `@for` are the modern, recommended way — `*ngIf` / `*ngFor` are good to recognize in existing codebases.

3 · ATTRIBUTE DIRECTIVES

ngClass & ngStyle — change how an element looks

```
<!-- toggle CSS classes based on conditions -->  
<div [ngClass]="{ active: isActive, disabled: isDisabled }">  
  Status box  
</div>  
  
<!-- apply inline styles dynamically -->  
<div [ngStyle]="{ color: isError ? 'red' : 'green' }">  
  Message  
</div>
```

Both change how an element **looks**, without adding or removing it from the page — that's what makes them attribute directives.

A custom hover-highlight directive

Write the shared behavior once as a Directive, then reuse it anywhere with a simple attribute — the same "don't repeat yourself" idea as Services, applied to DOM behavior.

1 · Generate

```
$ ng generate directive directives/highlight
```

2 · Write the logic

```
@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {
  private el = inject(ElementRef);

  @HostListener('mouseenter')
  onMouseEnter() { this.el.nativeElement.style.backgroundColor = 'yellow'; }

  @HostListener('mouseleave')
  onMouseLeave() { this.el.nativeElement.style.backgroundColor = ''; }
}
```

3 · Use it anywhere

```
<p appHighlight>Hover over me!</p>
<div appHighlight class="card">This whole card highlights too.</div>
```

- `selector: '[appHighlight]'` — square brackets mean it's used as an attribute, e.g. `<p appHighlight>`.
- `ElementRef` gives direct access to the actual HTML element it's attached to.
- `@HostListener('mouseenter')` runs automatically when that browser event fires — no manual event binding needed.

5 · BEST PRACTICES

When (and how) to write one

- Create a custom directive only when the **same** small behavior is needed on multiple, unrelated elements.
- Keep a directive focused on **one responsibility** — don't turn it into a mini-component.
- Prefer built-in directives (`@if`, `@for`, `ngClass`) whenever they already solve the problem.